

Connected to the server!
Enter your name:

60

Submit

Start Game

divider

網路程式設計期末專題 英文詞彙量比賽

資工二甲 1122918 林起尼

目錄

- 作品簡介
- 程式架構
- 所用到的所學技術
- 所用到的自學新技術
- 所遇到困難
- 程式碼說明
- 心得

作品簡介

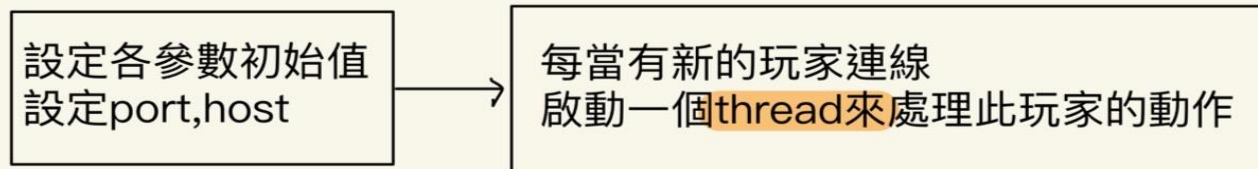
- 這是一個可以讓多人同時比賽英文詞彙量的遊戲。
- 每個人可以在時間內(60秒)去針對題目給的一個英文字母，打出自己所知道由這個字母開頭的單字。
- 當每個玩家都結束後，會廣播給大家每個連線玩家的排名。
- 得分:1.此單字是合法單字(英文字典有的)2.不是重複的。
- 違規(不計分):1.沒有這個單字2.非此次題目規定的開頭單字3.玩家在該回合已經輸入過的。

程式架構(server)

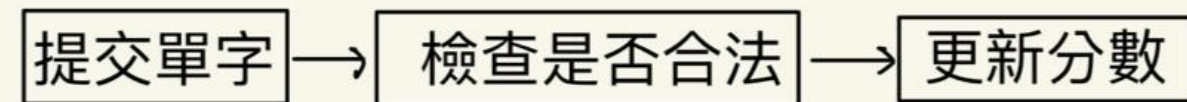
處理玩家連線，管理遊戲

-> 選字母、驗證單字、更新分數、廣播排名

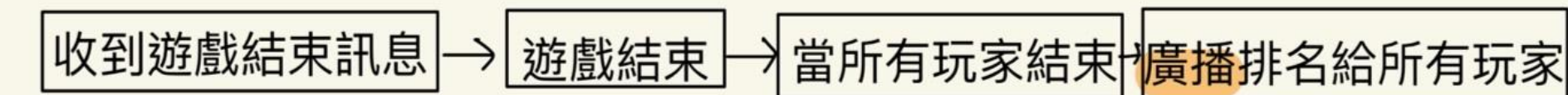
遊戲前



遊戲時



遊戲結束



程式架構(client)

提供界面，處理玩家輸入與倒數計時，並與伺服器連線。

遊戲前

用client sd 連到server → 顯示server發送過來的訊號

遊戲時

玩家按下start 開始遊戲並開始倒計時 → 點擊submit 把玩家單字交給server

遊戲結束

遊戲結束傳結束的訊息給server

所用技術(本學期所學)

1.建立的是TCP的連線

```
def start_server():  
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)#使用tcp  
    server_socket.bind((HOST, PORT))  
    server_socket.listen(5)  
    print("Server started... Waiting for connections.")  
  
    while True:  
        client_socket, addr = server_socket.accept()  
        threading.Thread(target=handle_client, args=(client_socket, addr)).start()
```

(綁定的host，port事先定義)

```
HOST = '127.0.0.1'  
PORT = 12345
```

2.用thread多工:每個client都有一個thread去專門處理

```
threading.Thread(target=handle_client, args=(client_socket, addr)).start()
```

所用技術(本學期所學)

3.用廣播，廣播給所有玩家最終排名

```
# 廣播
def broadcast_ranking(client_socket=None):
    # 排序得分
    sorted_players = sorted(players.items(), key=lambda x: x[1], reverse=True)
    ranking_message = "-----Final Rankings-----\n"
    for idx, (name, score) in enumerate(sorted_players, start=1):
        ranking_message += f"{idx}. {name}: {score} points\n"

    print(f"{ranking_message}")

    for player_name, client_socket in players_sockets.items():
        try:
            client_socket.send(ranking_message.encode()) # 發送排名給每個玩家
        except Exception as e:
            print(f"Error broadcasting ranking to {player_name}: {e}")
```

所用技術(額外學習到的)

- PyEnchant 字典檢查:透過`enchant.Dict("en_US")`建立英語字典物件，在遊戲過程驗證玩家輸入的單字是否為合法英文單字。
- Tkinter GUI:文字區呈現伺服器訊息，包括介面的設計，把訊息在訊息框內顯示給玩家看。
- 倒數計時:

1.`window.mainloop()`:Tkinter會進入一個迴圈，有事件就觸發對應的回呼函式，沒事件就等，這個迴圈一直跑到視窗被關閉為止。

```
window.mainloop()
```

2.`window.after(delay_ms, callback, *args)`

-`delay_ms`：延遲毫秒數（`1000 = 1 秒`）。

-`callback`：時間到後要執行的函式。

-`args`：傳給 `callback` 的參數（可省略）。

Ex.每隔一秒去呼叫原函式`update_countdown`

```
window.after(1000, update_countdown)
```

遇到的困難

- 原本在client，server兩端都有設倒數計時，但會遇到他們不同步的問題，還有要傳給client端讓他在訊息格顯示的資訊不知道為什麼收不到，導致無法每次玩家傳單字就更新分數，所以後來想說我就在client倒數計時，然後時間到後，在跟server端要我的總成績，還有等大家都結束後，要總成績的排名。
- 假設同時三個玩家連入，開始玩遊戲，本來的做法沒有設定讓每個人題目不同，會導致可能A，B都是一樣的題目，然後就會導致如果A打過這個單字，B再打就會不計分，想到最直覺的解覺方法就是給每個玩家不同的題目

程式碼說明(server)

先啟動server，然後等待client連入->連入後每個client分別用一個thread處理他(thread呼handle_client)

```
def start_server():
    server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)#使用tcp
    server_socket.bind((HOST, PORT))
    server_socket.listen(5)
    print("Server started... Waiting for connections.")

    while True:
        client_socket, addr = server_socket.accept()
        threading.Thread(target=handle_client, args=(client_socket, addr)).start()
```

程式碼說明(server)

Handle_client會處理每個玩家連入後的動作

```
def handle_client(client_socket, addr):
    print(f"New connection: {addr}")

    # 讓玩家輸入名字
    client_socket.send("Enter your name: ".encode())
    player_name = client_socket.recv(1024).decode().strip()
    players[player_name] = 0 # 初始化玩家分數

    # 玩家加入字典
    players_sockets[player_name] = client_socket
    client_socket.send(f"Hello, {player_name}! Please wait for the game to start...\n".encode())

    # 等待「開始遊戲」
    start_signal = client_socket.recv(1024).decode().strip()
    if start_signal == "start":
        # 隨機選擇一個字母
        if available_letters:
            letter = random.choice(available_letters)
            available_letters.remove(letter) # 確保每個玩家有不同字母
        else:
            letter = random.choice('abcdefghijklmnopqrstuvwxyz') # 字母池空->隨機選字母
        players_letters[player_name] = letter # 儲存玩家的字母
        client_socket.send(f"The letter is: {letter.upper()}\n".encode())
```

程式碼說明(server)

```
# 開始遊戲(接收輸入+處理)
while game_time_left > 0:
    word = client_socket.recv(1024).decode().strip().lower() # 轉小寫
    print(f"Received word from {player_name}: {word}")
    if not word:
        break

    # 檢查單字
    if word.startswith(letter) and d.check(word) and word not in used_words:
        used_words.add(word)
        players[player_name] += 1
        client_socket.send(f"Good job! You scored 1 point. Your current score: {players[player_name]}\n".encode())
    else:
        client_socket.send(f"Invalid word or repeated word. Your current score: {players[player_name]}\n".encode())

# 收到client的end訊息->結束遊戲
if word == "end_game":
    print(f"End game signal received from {player_name}")
    break
```

程式碼說明(server)

廣播排名給所有玩家

```
def broadcast_ranking(client_socket=None):
    # 排序得分
    sorted_players = sorted(players.items(), key=lambda x: x[1], reverse=True)
    ranking_message = "-----Final Rankings-----\n"
    for idx, (name, score) in enumerate(sorted_players, start=1):
        ranking_message += f"{idx}. {name}: {score} points\n"

    print(f"{ranking_message}")

    for player_name, client_socket in players_sockets.items():
        try:
            client_socket.send(ranking_message.encode()) # 發送排名給每個玩家
        except Exception as e:
            print(f"Error broadcasting ranking to {player_name}: {e}")
```

程式碼說明(client)

client連上server，且如果server有傳訊息來的話把它顯示在訊息框

```
# 客戶端連接伺服器
def connect_to_server():
    global client_socket
    client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

    try:
        client_socket.connect((HOST, PORT))
        display_message("Connected to the server!\n")
    except Exception as e:
        display_message(f"Connection failed: {e}\n")
        return

# 接收伺服器訊息
while True:
    msg = client_socket.recv(1024).decode()
    if msg:
        display_message(msg)
    else:
        break
```

程式碼說明(client)

處理玩家輸入的單字->連線後把輸入傳給server去判斷

```
def handle_input():
    word = entry.get().strip().lower()
    if word and client_socket: # 確保連線已建立
        try:
            client_socket.send(word.encode()) # 發送輸入到server
            entry.delete(0, tk.END)
        except Exception as e:
            display_message(f"Error sending message: {e}\n")
```

程式碼說明(client)

倒數計時，每隔一秒呼叫自己(window.after)，當結束後傳結束訊息給server讓他給出總成績

```
# 倒數計時顯示
def update_countdown():
    global time_left
    if time_left > 0:
        countdown_label.config(text=str(time_left))
        time_left -= 1
        window.after(1000, update_countdown) # 每秒更新一次倒數計時
    else:
        countdown_label.config(text="0") # 當時間結束顯示 0
        submit_button.config(state=tk.DISABLED) # 倒數結束關掉submit按鈕(不給按)

        # 要求顯示總成績
        client_socket.send("end_game".encode())

# 開始倒數計時
def start_countdown():
    global time_left
    time_left = 60 # 初始倒數時間
    update_countdown() # 開始倒數
```

心得

- 這次算是第一個自己完成的project，沒有同學可以討論的情況下，遇到的困難都得自己摸索，蠻不簡單的，會想到這個遊戲是因為自己英文蠻爛的，原本想做的一些都想不到怎麼做，所以做了這個，想說無聊也可以練練英文，畢竟有時候背了很多單字也不一定在需要的時候想得到，另外程式上除了學到上課老師說的thread在python上的用法，還有額外發現竟然有檢查拼音的模組，還蠻實用的，也因為期末project更知道上課的觀念怎麼用在實作，一開始還很擔心做不出來，除了做出東西以外，還要讓整體看起來比較舒服，在非常多次修改後終於成功了。